



ШПАРГАЛКА QA

ОСНОВЫ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Автор: Евгений Гусинец

ТГ-канал: QA♥4Life



БЛОК 1

ЧТО ТАКОЕ ТЕСТИРОВАНИЕ?

1

Определение · Необходимость · Парадигмы · Навыки · Роли QA/QC

СОДЕРЖАНИЕ БЛОКА 1

- 1.1** **Определение тестирования** ●
 Официальное определение ISTQB · ключевые слова · Static vs Dynamic
- 1.2** **Почему тестирование необходимо** ●
 Природа ошибок ПО · стоимость дефекта $\times 1 \rightarrow \times 1000$ · катастрофы · Shift-Left
- 1.3** **Тестирование $\neq$ поиск ошибок** ●
 Старая vs. новая парадигма · Positive / Negative Testing · Exhaustive Testing
- 1.4** **Как перейти к тестированию** ●
 Навыки QA 2025–2026 · карта Junior / Middle / Senior · инструменты
- 1.5** **Разница: QA / QC / Testing** ●
 Иерархия Testing $\subset$ QC $\subset$ QA · сравнение по 7 критериям · обязанности ролей

"Тестирование — это не фаза. Это образ мышления."

— Принцип современного QA

ОПРЕДЕЛЕНИЕ (ISTQB / ГОСТ Р 56920)

Тестирование (Testing) — процесс, охватывающий все активности жизненного цикла ПО (как статические, так и динамические), связанные с:

- планированием тест-активностей
- подготовкой тестовых условий и тест-кейсов
- выполнением тестов
- оценкой программного продукта и сопутствующих артефактов

Цели тестирования:

- ① Проверка соответствия продукта заявленным требованиям
- ② Демонстрация пригодности продукта для использования по назначению
- ③ Обнаружение дефектов

СВОИМИ СЛОВАМИ

Тестирование — это систематическая проверка того, что программный продукт работает так, как ожидается, и выявление мест, где он работает иначе.

СТАТИЧЕСКОЕ

Без запуска кода.
Просмотр требований, документации, кода (code review, инспекции, walkthroughs).

ДИНАМИЧЕСКОЕ

С запуском кода.
Выполнение тест-кейсов, проверка поведения системы в рантайме.

КЛЮЧЕВЫЕ СЛОВА §1.1

Статическое

Динамическое

Оценка

Дефекты

Требования

⚠ Тестирование ≠ Отладка (Debugging)

Тестирование — обнаруживает симптомы (что не работает). Отладка (Debugging) — находит и устраняет первопричину дефекта. Это разные активности, которые выполняют разные роли.

ГЛОССАРИЙ §1.1

Ошибка человека (Error)

Действие человека, приводящее к неправильному результату. Разработчик неверно понял требование или допустил опечатку в коде.

Дефект / Баг (Defect / Bug / Fault)

Исъян в компоненте или системе, способный привести к сбою. Результат ошибки человека, зафиксированный в артефакте (коде, документе).

Отказ системы (Failure)

Отклонение поведения компонента или системы от ожидаемого. Проявляется в рантайме при выполнении дефектного кода.

Тест-кейс (Test Case)

Набор предусловий, входных данных, действий, ожидаемых результатов и постусловий, разработанный для конкретной тест-цели.

Тест-основа (Test Basis)

Совокупность документов (требования, дизайн, код, стандарты), на основе которых разрабатываются тест-кейсы.

Тест-оракул (Test Oracle)

Источник для определения ожидаемого результата теста: спецификация, предыдущая версия системы, экспертное суждение.

ЦЕПОЧКА ДЕФЕКТОВ



1.2 ПОЧЕМУ ТЕСТИРОВАНИЕ НЕОБХОДИМО

Природа ошибок · Стоимость дефекта · Реальные катастрофы · Shift-Left

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

Программное обеспечение создаётся людьми → люди ошибаются по природе своей.

Ошибка человека (Error) → дефект в коде (Defect) → сбой системы в рантайме (Failure).

Именно поэтому тестирование → обязательная часть разработки, а не опциональная активность.

СТОИМОСТЬ ИСПРАВЛЕНИЯ ДЕФЕКТА

Чем позже обнаружен дефект — тем дороже его исправление

Требования (Requirements)		× 1
Исправить дешевле всего		
Дизайн (Design)		× 5
Разработка (Development)		× 10
Тестирование (Testing)		× 20
Ещё можно исправить до релиза		
Продакшн (Production)		× 100 — 1000
Критично: пользователи уже видят баг		

 **Вывод:** Найти дефект на этапе требований обходится в 100–1000 раз дешевле, чем в продакшне. Принцип Shift-Left Testing — тестируй раньше.

КАТАСТРОФЫ ИЗ-ЗА ОШИБОК ПО

Реальные случаи из истории — цена программных ошибок



Ракета Ariane 5 (1996)

Переполнение 64-битного числа при записи в 16-битную переменную. Система аварийной защиты уничтожила ракету через 37 секунд после старта.



Ущерб: \$370 млн



04.06.1996



Mars Climate Orbiter (1999)

Команда Lockheed Martin передавала данные тяги в имперских единицах (фунт-сила-с), команда NASA ожидала метрические (Н-с). Зонд вышел на слишком низкую орбиту и сгорел.



Ущерб: \$327 млн



Сентябрь 1999



Therac-25 (1985–1987)

Ошибка синхронизации в медицинском радиационном ускорителе. Race condition при быстром вводе оператора. Пациенты получили смертельные дозы радиации в 100× норму.



Погибших: 3 человека, тяжело пострадавших: 3

В медицине, авиации и финансах программные ошибки = угроза жизни людей. Тестирование — это не формальность.

ПРИНЦИП СДВИГА ВЛЕВО — SHIFT-LEFT TESTING



БЕЗ Shift-Left:

Баги попадают в продакшн · Пользователи находят дефекты · Репутационные потери · Исправление стоит ×100–1000

С Shift-Left:

Тесты пишутся параллельно с требованиями · QA участвует с первого дня · Дефекты дешёвы · Качество встроено в процесс

Positive Testing (Test to Pass) · Negative Testing (Test to Fail) · Покрывшие сценариев

☑ POSITIVE TESTING

Test to Pass · «Счастливый путь»

**ПОЗИТИВНОЕ ТЕСТИРОВАНИЕ**

Проверка того, что система **КОРРЕКТНО РАБОТАЕТ** при валидных входных данных и ожидаемых условиях использования. Имитируем поведение обычного пользователя.

Вопрос: "Работает ли система как задумано?"

- Входные данные — валидные, в допустимом диапазоне
- Условия — штатные, предусмотренные требованиями
- Ожидание — система выполняет действие успешно

💡 "Happy Path" — сценарий довольного пользователя

✗ NEGATIVE TESTING

Test to Fail · «Пути ошибок»

**НЕГАТИВНОЕ ТЕСТИРОВАНИЕ**

Проверка того, что система **КОРРЕКТНО ОБРАБАТЫВАЕТ** невалидные данные, ошибки и граничные условия. Имитируем поведение злоумышленника, неопытного пользователя или сбой среды.

Вопрос: "Не сломается ли система при неправильном вводе?"

- Входные данные — невалидные, пустые, выходящие за границы
- Условия — нештатные, ошибочные, экстремальные
- Ожидание — система отображает корректное сообщение об ошибке

💡 "Sad Path / Error Path" — сценарий ошибки или злоупотребления

ПРИМЕРЫ ДЛЯ ТИПИЧНЫХ СЦЕНАРИЕВ QA

Поле формы регистрации: email, пароль, возраст. REST API: POST /register

Сценарий	☑ Positive (ожидаемый Pass)	✗ Negative (ожидаемый Fail)
Email	user@example.com → 201 Created, письмо отправлено	user@example (без домена), пустая строка "", user@, @@example.com → 400 Bad Request + сообщение об ошибке
Пароль	P@ssw0rd123 (8 символов, буквы+цифра+специальный символ) → 201 Created	"123" (слишком короткий), "" (пустой), "password" (без специальных символов) → 422 Validation Error
Возраст	25 (допустимый диапазон 18–99) → поле принято	17 (меньше минимума), 150 (больше максимума), -1 (отрицательное), "двадцать пять" (строка вместо числа) → ошибка валидации
Загрузка файла	photo.jpg, 2 MB, JPEG — успешная загрузка, превью отображается	exploit.exe (неверный формат), 50 MB (превышение лимита), пустой файл 0 KB → соответствующее сообщение об ошибке
Авторизация	Верный логин + верный пароль → 200 OK, токен выдан	Верный логин + неверный пароль → 401 Unauthorized. Несуществующий логин → 404 или 401. 6-я попытка подряд → блокировка аккаунта
Поиск	"iPhone 15" → список товаров, сортировка работает	"" (пустой запрос) → подсказка или ошибка, не краш. SQL: "'; DROP TABLE--" → экранируется, XSS: "<script>alert(1)</script>" → отображается как текст
Числовое поле	Количество: 3 → товар добавлен 3 раза	0 (ноль), -5 (отрицательное), 9999999 (overflow), 1.5 (дробное для целочисленного поля), буква "к" → валидация отклоняет
API: токен авторизации	Bearer eyJhbGc... (валидный JWT) → 200 OK, данные возвращены	Без заголовка Authorization → 401. Истёкший токен → 401. Случайная строка вместо JWT → 401 или 400

КОГДА И КАК ПРИМЕНЯТЬ**📋 НАЧНИ С POSITIVE**

Всегда сначала убедись, что "счастливый путь" работает. Если основной сценарий сломан — negative тестировать бессмысленно.

⚡ ДОБАВЛЯЙ NEGATIVE

После подтверждения позитивных сценариев переходи к граничным значениям, пустым полям, неверным форматам и нагрузочным сценариям.

⚖️ БАЛАНС 70/30

Типичное соотношение в проекте: 70% позитивных тест-кейсов + 30% негативных. При высоких требованиях безопасности — 50/50.

🔗 СВЯЗЬ С ТЕХНИКАМИ

Negative тесты = основа техник: Граничные значения (BVA), Классы эквивалентности (ECP), Тестирование ошибок. Positive = Happy Path, Smoke Test.

🎯 ПРАВИЛО ПОЛНОГО ПОКРЫТИЯ

Для каждого тестируемого условия (поле, функция, API-метод) должны существовать ОБА типа тест-кейсов. Только положительные тесты создают ложную уверенность в качестве продукта. Только отрицательные — не проверяют основной функционал.

ПОЛНОЕ ПОКРЫТИЕ = Positive Test Cases + Negative Test Cases

1.4 НАВЫКИ QA-ИНЖЕНЕРА 2025–2026

Актуальная карта компетенций: Junior · Middle · Senior

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

ТЕОРИЯ И ОСНОВЫ

Must Have — для всех уровней

- Виды тестирования: функциональное, регрессионное, интеграционное, Smoke, Sanity, приёмочное
- Техники тест-дизайна: классы эквивалентности, граничные значения (BVA), таблицы решений, попарное тестирование (Pairwise)
- Понимание SDLC / STLC, Agile (Scrum, Kanban)
- Написание тест-кейсов, тест-планов, чек-листов
- Баг-репорты: шаги воспроизведения, Severity, Priority
- Верификация и валидация, уровни тестирования

ИНСТРУМЕНТЫ JUNIOR

Must Have Junior / Junior+

- Баг-трекинг: Jira, YouTrack, Redmine
- Документация: Confluence, Notion
- Тест-менеджмент: Zephyr, Allure TestOps, TestRail
- API-тестирование: Postman, Swagger / OpenAPI
- SQL: SELECT, INSERT, UPDATE, DELETE, JOIN (INNER, LEFT, RIGHT), GROUP BY, WHERE, подзапросы
- UI: DevTools браузерера, BrowserStack, Figma (чтение макетов)
- Контроль версий: Git (базовые команды)
- Мобильное: Android Studio, Xcode, Charles Proxy / Proxyman

АВТОМАТИЗАЦИЯ

Must Have Middle — обязательно на рынке 2025–2026

- Языки: Python, JavaScript / TypeScript, Java
- UI-автотесты: Selenium, Playwright, Cypress
- API-автотесты: REST-assured, Requests (Python)
- BDD: Cucumber, Behave (Given / When / Then)
- TDD: принцип, JUnit, pytest
- CI/CD: Jenkins, GitLab CI, GitHub Actions, TeamCity
- Docker: поднять тестовое окружение
- Мониторинг: Kibana / Elastic, Grafana
- Linux: tail, grep, cat, ls, find

ADVANCED & AI (ТРЕНД 2026)

Middle+ и Senior

- Kafka, RabbitMQ, Redis — асинхронное тестирование
- Kubernetes — чтение логов подов
- Cloud: AWS / Azure / GCP (базовое)
- AI-инструменты: TestSigma, ACCELQ
- Data-driven тестирование: Python / SQL / Tableau
- Микросервисная архитектура

МАТРИЦА УРОВНЕЙ

✓ Must Have • Желательно — Не нужно

НАВЫК	JUN	MID	SEN
Теория тестирования	✓	✓	✓
SQL базовый	✓	✓	✓
SQL сложный (JOIN, GROUP BY)	Желательно	✓	✓
REST API / Postman	✓	✓	✓
Git (базовые команды)	✓	✓	✓
DevTools / Figma	✓	✓	✓
Мобильное + снифферы	Желательно	✓	✓
Kibana / Grafana / логи	Желательно	✓	✓
Автоматизация Selenium/Playwright	Желательно	✓	✓
Python / JS / Java	Желательно	✓	✓
CI/CD Jenkins / GitLab CI	—	Желательно	✓
Docker	—	Желательно	✓
Kafka / RabbitMQ	—	Желательно	✓
Kubernetes / Cloud	—	—	Желательно
AI-инструменты тестирования	Желательно	Желательно	✓

⚠ Рынок 2025–2026: Junior без SQL, API и Git практически не берут. Middle обязан владеть хотя бы одним фреймворком автоматизации.

SOFT SKILLS —
Must Have для всех

Аналитическое
мышление

Коммуникация

Правильные вопросы
к требованиям

Приоритизация

Работа
в команде

1.5 РАЗНИЦА: QA / QC / ТЕСТИРОВАНИЕ

Иерархия концепций · Детальное сравнение · Обязанности на проекте

ОСНОВЫ ТЕСТИРОВАНИЯ ПО

ИЕРАРХИЯ

Testing $\subset$ QC $\subset$ QA

QA

Quality Assurance

QC

Quality Control

Testing

Тестирование
(Testing)

Контроль качества продукта

Обеспечение качества процессов

QA — проактивный подход.
Предотвращает появление дефектов
путём улучшения процессов.

QC — реактивный подход.
Обнаруживает несоответствия
в готовом продукте или его части.

Testing — реактивный подход.
Непосредственное выполнение
проверок продукта.

ДЕТАЛЬНОЕ СРАВНЕНИЕ ПО 7 КРИТЕРИЯМ

КРИТЕРИЙ	QA	QC	Testing
Расшифровка	Quality Assurance	Quality Control	—
Суть	Обеспечение качества Процессы разработки	Контроль продукта Выполнение проверок	Контроль продукта Выполнение проверок
Тип подхода	Проактивный — предотвращает	Реактивный — находит	Реактивный — фиксирует
Когда работает	На всех этапах SDLC	После создания артефактов	На этапе выполнения тестов
Объект	Процессы разработки и тестирования	Продукт и итоги тестирования	Продукт как результат
Отношение к процессу	Выстраивает процессы	Подмножество QA	Подмножество QC
Меры	Превентивные	Корректирующие	Выявление несоответствий

ОБЯЗАННОСТИ НА ПРОЕКТЕ

● QA Engineer

- Планирование и оптимизация процессов разработки
- Работа с документацией и требованиями
- Коммуникация с заказчиком
- Оценка рисков проекта
- Внедрение и актуализация методик тестирования
- Анализ работы тестировщиков и QC-специалистов
- Предотвращение появления дефектов

● QC Specialist

- Анализ результатов тестирования
- Предоставление информации о текущем уровне качества
- Работа с билдами в процессе разработки
- Проработка тест-кейсов и их результатов
- Составление отчётов о качестве

● Test Engineer

- Составление тест-кейсов и чек-листов
- Выполнение тест-кейсов согласно тест-плану
- Локализация дефектов
- Составление баг-репортов
- Взаимодействие с разработчиками при верификации исправлений
- Фокус непосредственно на тестировании



ВАЖНО НА ПРАКТИКЕ:

В небольших командах все три роли нередко совмещает один специалист с тайтлом "QA Engineer" или "Tester". Чёткое разграничение QA / QC / Testing — удел зрелых организаций и крупных проектов.



ЗАЧЕМ ЗНАТЬ ПРИНЦИПЫ

7 принципов — это фундамент профессионального мышления QA-специалиста, закреплённый в стандарте ISTQB CTFL 4.0. Они объясняют природу тестирования и предотвращают типичные ошибки новичков.

ISTQB CTFL 4.0

1 Тестирование показывает наличие дефектов

Testing shows presence of defects

Тестирование доказывает НАЛИЧИЕ дефектов, но НИКОГДА не может доказать их полное отсутствие. Прошедшие тесты ≠ баги отсутствуют.

✦ ПРИМЕР: 500 тест-кейсов пройдено. Завтра пользователь нашёл баг в сценарии, который не был протестирован.

ОШИБКА: "Все тесты прошли — продукт готов!"

2 Исчерпывающее тестирование невозможно

Exhaustive testing is impossible

Охватить все комбинации входных данных, условий и последовательностей действий физически невозможно. Нужна приоритизация по риску.

✦ ПРИМЕР: Поле "возраст" (0–150) × поле "имя" (любая строка) × 5 браузеров × 3 ОС = миллиарды комбинаций. Тестируем только критичные.

РЕШЕНИЕ: Risk-based testing · BVA · Pairwise

3 Раннее тестирование экономит время

Early testing saves time and money

Дефект на стадии требований исправляется в 100–1000 раз дешевле, чем в продакшне. Shift-Left = тестировщик участвует с первого дня.

✦ ПРИМЕР: QA замечает противоречие в User Story до написания кода — экономия 2 дней разработки. Vs. тот же баг в продакшне — 2 недели + деньги.

ПРИНЦИП: Shift-Left Testing

4 Скопление дефектов

Defect clustering (Pareto principle)

80% дефектов концентрируются в 20% модулей. Дефекты не распределены равномерно — они кластеризуются в "проблемных" местах кода.

✦ ПРИМЕР: На проекте e-commerce 70% багов — в модуле корзины и оплаты. QA уделяет им повышенное внимание при регрессии.

ВЫВОД: Усиливай тестирование рискованных модулей

5 Парадокс пестицида

Pesticide paradox

Одни и те же тесты, повторяемые снова и снова, теряют способность находить новые дефекты. Тест-сьют необходимо регулярно обновлять и дополнять.

✦ ПРИМЕР: Регрессионный набор не обновлялся 6 месяцев. Новая фича внесла баги в старый модуль, но ни один старый тест это не поймал. Нужны новые тест-кейсы!

ДЕЙСТВИЕ: Регулярно ревьюить и пополнять тест-сьют.

6 Тестирование зависит от контекста

Testing is context-dependent

Нет универсального подхода. Тестирование медицинского ПО (ISO 62304) ≠ тестирование мобильной игры. Методы, глубина и стандарты определяются контекстом.

✦ ПРИМЕР: ПО для управления самолётом — требует формальные методы, строгая документация, многоуровневая верификация. Игровое приложение — экспресс-тесты, UX-фокус, A/B.

ПРИНЦИП: One size does NOT fit all в тестировании.

7 Заблуждение об отсутствии ошибок

Absence-of-errors fallacy

Нахождение и исправление дефектов бесполезно, если система не отвечает потребностям пользователей или не пригодна для использования по назначению.

✦ ПРИМЕР: Система бухгалтерского учёта без единого бага — но интерфейс настолько сложен, что бухгалтеры отказываются ей пользоваться. Качество ≠ только отсутствие багов.

КЛЮЧ: Validation важна не меньше Verification.

ТИПИЧНЫЕ ЗАБЛУЖДЕНИЯ НОВИЧКОВ

✗ Заблуждение

✓ Что на самом деле (принцип)

"Если все тесты прошли — продукт без багов"

Тестирование лишь снижает риск. Принцип №1.

"Нам нужно протестировать всё!"

Исчерпывающее тестирование невозможно. Принцип №2.

"Старые тесты защитят нас от новых багов"

Тест-сьют нужно обновлять. Принцип №5 — парадокс пестицида.

"Нет багов = качественный продукт"

Продукт должен решать задачу пользователя. Принцип №7.